5.1 Cron定时任务入门

> **生成模型**：OpenAI GPT-5.3 Codex (`openai/gpt-5.3-codex`)
> **Token 消耗**：输入 ~22k tokens，输出 ~5.6k tokens（估算）

前面你已经看到 OpenClaw 会聊天、会调用工具，但它默认还是“你说一句，它做一句”。Cron 的意义在于把它变成“你不说，它也会按计划做事”。很多人第一次配完 Cron 任务，第二天起床看到机器人已经把天气和摘要发来了，会直观感受到：这不只是聊天机器人，而是一套自动值班系统。

这一节从使用者视角把 OpenClaw 的 Cron 跑通：Cron 是什么、表达式怎么读、任务如何创建管理、结果如何投递、常见坑怎么避开。我们不做第 26 章源码深挖，只讲马上能用起来的知识。

5.1.1 Cron 到底是什么

Cron 最早来自 Unix/Linux 世界，本质是一个**定时调度器**（Scheduler）。你给它一条时间规则，它就会在对应时刻触发任务。

在传统服务器里，这个任务通常是一段 shell 命令；在 OpenClaw 里，这个任务更像“发起一轮 Agent 执行”。也就是：

- 传统 Cron：到点执行脚本
- OpenClaw Cron：到点唤醒 Agent，Agent 可以继续调用工具、整合信息、产出结果并投递

这就是为什么 OpenClaw 的 Cron 不只是“定时提醒”。它可以做真正的自动化流程，比如“每天 8 点抓天气 -> 生成建议 -> 发 Telegram”，或者“每 15 分钟巡检服务 -> 异常就解释并给修复建议 -> 发 Slack”。

> **衍生解释 - 调度（Scheduling）**
>
> “调度”就是安排任务在什么时机、以什么顺序运行。它不负责任务内容有多聪明，只负责让任务在正确时间被执行。Cron 就是一种经典调度方式。

5.1.2 Cron 表达式：5 列就够你上手

OpenClaw 常用 5 段 Cron 表达式：

```
分 时 日 月 周
```

也就是从左到右：分钟、小时、每月第几天、第几月、星期几。

最经典示例：

```
0 8 * * *
```

含义：**每天早上 8 点执行**。

再看几个你会经常用到的：

```
*/15 * * * *
```

每 15 分钟执行一次。

```
30 9 * * 1
```

每周一 9:30 执行。

```
0 22 * * 1-5
```

每个工作日（周一到周五）22:00 执行。

```
0 0 1 * *
```

每月 1 号零点执行。

常见符号说明：

- `*`： 任意值（每一个）
- `*/n`： 每 n 个单位一次
- `a-b`： 范围
- `a,b,c`： 枚举

新手最容易踩的坑是把“分”和“时”写反：

- `8 0 * * *` 是每天 **00:08**
- `0 8 * * *` 才是每天 **08:00**

写完 Cron 后建议用中文读一遍；读不顺，规则大概率有问题。

5.1.3 OpenClaw 一条定时任务由什么组成

你可以把一个 Cron 任务拆成四个问题：

1. **何时跑**（`schedule`）
2. **跑什么**（`payload.message`）
3. **在哪跑**（`sessionTarget`）
4. **结果送哪**（`delivery`）

对应到配置结构通常长这样：

```
{
  cron: {
    enabled: true,
    maxConcurrentRuns: 2,
    sessionRetention: "24h"
  },
  jobs: [
    {
      name: "Morning Weather",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "0 8 * * *",
        tz: "Asia/Shanghai"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "查询今天杭州天气，并给我通勤建议。"
      },
      delivery: {
        mode: "announce",
        channel: "telegram",
        to: "123456789"
      }
    }
  ]
}
```

你不一定每天都手写 JSON，CLI 也能创建任务；但你必须看得懂这套结构，后面排错和迁移都会用到。

5.1.4 session 选主会话还是隔离会话

Cron 任务在 OpenClaw 里最常见两种运行位置：

- `main` （主会话）
- `isolated` （隔离会话）

简单理解：

- **主会话**：任务会复用你日常聊天上下文，适合“持续跟进同一话题”
- **隔离会话**：每个任务在单独上下文跑，更干净，适合高频巡检和后台自动化

实践建议很直接：只要不是必须依赖当前聊天上下文，优先用 `isolated` 。这会减少上下文污染，也更容易复盘每次运行。

5.1.5 创建和管理 Cron 任务（CLI）

示例：每天 8 点发天气简报到 Telegram。

```
openclaw cron add   --name "Morning Weather"   --cron "0 8 * * *"   --tz "Asia/Shanghai
```

常用管理命令：

```
openclaw cron list
openclaw cron status
openclaw cron runs --id <job-id>
openclaw cron run <job-id> --force
```

建议形成一个小习惯：**新任务一律先 `run --force` 人工试跑一次**。别等真实触发时间到了才发现表达式或投递目标写错。

5.1.6 Delivery Plan：结果发给谁

Cron 任务“执行成功”不等于“自动化闭环成功”。你还要明确结果怎么送出去。OpenClaw 常见三种投递模式：

1) `announce`：发到聊天渠道

```
{
  delivery: {
    mode: "announce",
    channel: "slack",
    to: "channel:C0123456789",
    bestEffort: true
  }
}
```

适合人要看的通知，比如日报、巡检摘要、告警提示。

2) `none`：只执行，不主动外发

```
{
  delivery: {
    mode: "none"
  }
}
```

适合后台计算、预处理、静默巡检等不希望刷屏的任务。

3) `webhook` ： POST 到外部系统

```
{
  cron: {
    enabled: true,
    webhookToken: "cron-webhook-token"
  },
  jobs: [
    {
      name: "Daily Archive",
      schedule: { kind: "cron", expr: "0 22 * * 1-5" },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "生成工作日报摘要并输出结构化结果。"
      },
      delivery: {
        mode: "webhook",
        to: "https://ops.example.com/openclaw/daily-archive"
      }
    }
  ]
}
```

适合机器对机器联动，比如告警系统、工单系统、数据流水线。

5.1.7 三个上手就能用的 Cron 示例

示例 A：每天 8 点天气推送

- Cron： `0 8 * * *`
- 含义：每天 8:00
- 用途：出门前一眼看完天气和通勤建议

示例 B：每 15 分钟服务巡检

- Cron： `*/15 * * * *`
- 含义：每 15 分钟
- 用途：轻量运维值班，异常时第一时间通知

示例 C：工作日 22:00 日报归档

- Cron： `0 22 * * 1-5`
- 含义：周一到周五每天 22:00
- 用途：把当天工作摘要自动归档到外部系统

这些场景看起来简单，但组合起来已经是一个“个人运维 + 个人信息中台”的雏形了。

5.1.8 常见坑与规避建议

- 时间写对了，**时区没写**：服务器在海外会直接错时，建议显式写 `tz`
- 任务描述太空：不要只写“检查系统”，要写检查项和输出格式
- 高频任务放主会话：上下文会迅速变脏，优先 `isolated`
- 投递目标用 `last` 但路由不稳定：多渠道场景建议写死 `channel + to`
- 一上来就高风险自动修复：先“自动诊断 + 人工确认”，再逐步放权

本节小结

Cron 让 OpenClaw 从“被动问答”进入“主动执行”。你真正要掌握的核心只有四件事：会读 Cron 表达式（如 `0 8 * * *` 是每天 8 点）、会选运行会话（优先 `isolated`）、会配结果投递（`announce` / `none` / `webhook`）、会在上线前用 `run --force` 验证。把这四件事用顺，自动化就不再是概念，而是每天都在替你省时间的基础设施。